

Stochastic optimization algorithms 2025

Home problems, set 1 (Python version)

General instructions. READ CAREFULLY!

Problem set 1 consists of three parts. Problems 1.1 and 1.3 are mandatory, Problem 1.2 is voluntary (but check the requirements for the various grades on the web page). After solving the problems, collect your answers, your programs, *and* your report (see below) in *one* .zip (or .7z) file which, when opened, generates one main folder containing your report and additional subfolders for each problem (e.g. Problems 1.1 and 1.3) in the assignment. In problems that do not involve computer programming (e.g. Problem 1.2, in this case), no folder is needed.

You should provide a single report (for Problems 1.1-1.3, i.e. the whole set) in the form of a PDF file (Note: *Only* this format is accepted). The required sections of the report are specified in the individual problems below. In the case of analytical problems, make sure to include *all the relevant steps of the calculation* in your report, so that the calculations can be followed. Providing only the answer is *not* sufficient. Whenever possible, use symbolical calculations as far as possible, and introduce numerical values only when needed. You should write the report yourself (without help from chatbots of any kind) on a computer, preferably using LaTeX (see www.miktex.org). Scanned, handwritten pages are *not* allowed.

In *all* problems requiring programming, use Python (preferably v3.13.5; see also the course web page). The Python program for each problem should be placed in the folder for the problem in question. Do not include unnecessary files, e.g., unused Python files. It should *not* be necessary to edit the programs, move files etc. Furthermore, when writing Python programs, you should make sure to follow the coding standard (available on the web page). You may, however, hard-code *parameters*, as is indeed done in some of the skeleton files from which you should start. Programs that do not function, or require editing to function, or deviate significantly from the coding standard, or otherwise do not follow the requirements given above, will be returned, resulting in a deduction of points. The maximum number of points for problem set 1 is 10. Incorrect solutions will be returned for correction *only* in cases where the mandatory requirements have not been met, so please make sure to check your solutions and programs carefully before submitting the (single) compressed file (.zip or .7z) in Canvas.

You may, of course, discuss the problems with other students. However, each student *must* hand in his or her *own* solution. Note that a plagiarism check will be carried out, in which both your report and your code are checked against reports and code from other students (both from this year and earlier years). In obvious cases of plagiarism, points will be deducted from all students involved.

NOTE: Don't forget to write your name *and* civic registration number on the front page of the report! Make sure to keep copies of the files that you hand in! Good luck!

(Strict) deadline: 20250923, 23.59.59

Problem 1.1, 3p, Penalty method (Mandatory)

In this problem, we shall use the penalty method (see pp. 30-33 in the course book) to find the minimum of the function

$$f(x_1, x_2) = (x_1 - 1)^2 + 2(x_2 - 2)^2, \quad (1)$$

subject to the constraint

$$g(x_1, x_2) = x_1^2 + x_2^2 - 1 \leq 0. \quad (2)$$

You should now do the following steps:

1. Define (and specify clearly, in your report, as a function of x_1, x_2 , and μ) the function $f_p(\mathbf{x}; \mu)$, consisting of the sum of $f(x_1, x_2)$ and the penalty term.
2. Next, compute (analytically) the gradient $\nabla f_p(\mathbf{x}; \mu)$, and include it in your report. Make sure to include both the case where the constraints are fulfilled and the case where they are not.
3. Find (analytically) the unconstrained minimum (i.e. for $\mu = 0$) of the function, and include it in your report. This point will be used as the starting point for gradient descent.
4. Write a Python program for solving the unconstrained problem of finding the minimum of $f_p(\mathbf{x}; \mu)$ using the method of gradient descent. You *should* make use of the skeleton file available (`run_penalty_method.py`) on Canvas that, in turn, calls the other functions, generates and prints output etc. etc.). Here, you should set a suitable starting point (see above) and you may modify parameters (e.g. the values of μ or the tolerance parameter). The functions that you must implement (in the `run_penalty_method.py` file, for simplicity) are
 - (a) `run_gradient_descent` that should take the starting point $\mathbf{x}_0$ (as a vector with two elements), the value of μ , the step length (for gradient descent) η , and a threshold T (see below) as input, and carries out gradient descent until the (L2) norm of the gradient, $\|\nabla f_p(\mathbf{x}; \mu)\|$, drops below the threshold T . Use the unconstrained minimum as the starting point; see above.
 - (b) `compute_gradient` that should take as input the vector x (a vector with two elements, as a Python list) and μ , and returns (for any point $x \in \mathbf{R}^2$) the gradient of $f_p(\mathbf{x}; \mu)$ (a vector with two elements, also as a Python list). Note: You should hardcode the gradient in this method, i.e. you do not need to write a general method for finding the gradient. However, your method should make use of the analytical gradient, computed in Step 2 above. You should *not* use a numerical approximation of the gradient.

5. Run the program for a suitable sequence of μ values (specified in `run_penalty_method.py`)
Select a suitable (small) value for the step length η , and specify it clearly, along with the sequence of μ values, in your report. Example of suitable parameter values: $\eta = 0.0001$, $T = 10^{-6}$, sequence of μ values: 1, 10, 100, 1000. As output, the program gives the components of the vector x for the different μ values. You should include the table in your report; see below. Specify the values of x_1^* and x_2^* with 4 decimal precision. Do *not* just print the raw Python output (with many decimals, for example) in your report! You *must* also check that your results are reasonable, i.e. that the sequence of points appears to be convergent, for example by plotting the values of x_1 and x_2 as functions of μ . Note: Add at least one sentence in your report discussing the (estimated) convergence of these point sequences.

What to hand in You should hand in your Python program, with the methods described above (see Step 4 in the list). In your report, you should include a section named *Problem 1.1*, containing the calculations and results for Steps 1 - 3 in the list above, as well as the table with the output from your program, along with a brief discussion regarding the convergence of the point sequences, as described under Step 5 above. Make sure (where applicable) to include any relevant intermediate steps, not just writing down the final result! Omitting intermediate steps may result in the solution being returned, with resulting point deductions; see below.

Maximum number of points for this problem: 3p.

Maximum number of points if your solutions must be returned for correction: 1p

Problem 1.2, 3p, Constrained optimization (Voluntary)

Use the analytical method described on pp. 29-30 in the course book to determine the maximum and minimum values (as well as the corresponding values of x_1 and x_2) of the function

$$f(x_1, x_2) = 4x_1^2 + 2x_2^3, \quad (3)$$

over the closed set S defined by

$$x_1^2 + x_2^2 \leq 4. \quad (4)$$

When checking the boundary of S, use the Lagrange multiplier method. In your solution, make sure to list *all* critical points, both in the interior of S and on the boundary.

What to hand in For this problem you should *not* write any computer program. You should, however, include a section (*Problem 1.2*) in your report, where you include your full solution. You should describe the solution process clearly, showing all relevant intermediate steps.

Problem 1.3, 4p, Basic GA program (Mandatory)

a) In this problem, you will implement a genetic algorithm (GA) for finding the minimum of the function

$$g(x_1, x_2) = (1.5 - x_1 + x_1x_2)^2 + (2.25 - x_1 + x_1x_2^2)^2 + (2.625 - x_1 + x_1x_2^3)^2 \quad (5)$$

in the range $[-5, 5]$ (for both x_1 and x_2). You must use the skeleton files provided for this problem on the Canvas page. There are two main files, `run_single.py` for running the GA once and `run_batch.py` for making multiple runs and also obtaining some statistics. In `run_single.py` you may *only* modify those parameters marked *changes allowed*. The allowed changes in `run_batch.py` will be described in part (b) below. These two Python programs should access the function `run_function_optimization` in the module `genetic_algorithm.py` (for which there is also a skeleton file available). Now do the following:

Using the skeleton file (in which you may *not* change the interfaces) on the Canvas page, and making sure to follow the coding standard, write functions for...

1. ... initializing a population (`initialize_population`),
2. ... decoding a (binary) chromosome (`decode_chromosome`),
3. ... evaluating an individual (`evaluate_individual`),
4. ... selecting individuals with tournament selection (`tournament_select`),
5. ... carrying out crossover (`cross`),
6. ... carrying out mutations (`mutate`).
7. ... running the GA (`run_function_optimization`)

Versions of these functions have been implemented during the Programming introduction. However, for this problem you will make (some of) the functions more general, as follows:

initialize_population: This function should take the population size and the number of genes as input, and should return the entire population as a matrix of binary numbers (i.e. as in the Programming introduction).

decode_chromosome: This function should take as input (i) a (binary) chromosome in the form of a Python list, (ii) the number of variables that are to be extracted, and (iii) the maximum variable value. Let m denote the chromosome length and n the number of variables, and let $k = m/n$. The first k bits should be used when forming x_1 , the next k bits should be used for generating x_2 etc. Each variable should be decoded from the k bits according to Eq. (3.9) in the course book. You may assume that m and n have been chosen such that k is an integer.

evaluate_individual: This function should take the vector of variables ($\mathbf{x}$) as input (in the form of a Python list), and should return the (note!) *fitness value*, which in this case should be taken as $(g(x_1, x_2) + 1)^{-1}$, keeping in mind that $g(x_1, x_2)$ is clearly non-negative but might (perhaps) take the value 0. You should hardcode the function in `EvaluateIndividual` (i.e. you should *not* place the function in a separate file) and then also hardcode the computation of the fitness value. Check carefully that you enter the function $g(x_1, x_2)$ correctly!

tournament_select: This function should take as input (i) the vector of fitness values (from the most recently evaluated population) in the form of a Python list, (ii) the tournament selection parameter and (iii) the tournament size, and should return the index of the selected individual, using tournament selection. Note that the function should also handle cases where the tournament size is different from 2! See the description near the top of p. 50 in the course book.

cross: This function should take two chromosomes as input (each in the form of a Python list), carry out single-point crossover, and return a chromosome pair (i.e. as in the Programming introduction).

mutate: The `mutate` function should take as input (i) a chromosome (a Python list) and (ii) a mutation probability, and should return a mutated chromosome (i.e. as in the Programming introduction).

run_function_optimization This function corresponds to the main script (program) in the programming introduction, but it must now be turned into a function that returns the maximum fitness as well as the corresponding variable values.

After completing any Python function, you should preferably carry out a *unit test*, i.e. writing a simple wrapper that just provides suitable input to the function in question, and then make sure that the function generates correct output. (You should not hand in any of the unit tests, though).

When you have completed your program, experiment with some different values of the parameters that you are allowed to change, then select a set of parameters (which you may hardcode in `run_single.py`), and then carry out 10 runs of the algorithm, noting the values found, both for x_1 and x_2 and for the function $g(x_1, x_2)$ (defined above).

b) Since the goal of this problem is for you to familiarize yourselves with GAs, you should also make a parameter search that, in this problem, will concern only the mutation rate (normally, one would of course carry out an analysis involving several (or all) parameters). For this analysis, you should use the `run_batch.py` file. The skeleton file already carries out a batch of 100 runs with the mutation probability p_{mut} equal to 0.02 ($= 1/m$), and then prints some statistics. You should add code in `run_batch.py` so that the program makes several batch runs (in sequence) for different values of the mutation probability, namely 0.000, 0.005, 0.010, 0.020, 0.050, and 0.100. For every value of p_{mut} , 100 runs should be carried out. Then you should

tabulate and plot (and include in your report; see below) the (note!) *median* performance (= fitness value), as a function of the mutation probability p_{mut} , where the median (for each value of p_{mut}) is taken over the 100 runs. You are allowed to make the necessary changes in `RunBatch.m` so that the program carries out the runs just described and prints the necessary statistics, but make no other changes in that file, i.e., keep all other parameters as specified there.

c) Note that the GA provides a numerical *estimate* of the true minimum, but your goal is to find the exact minimum. Thus, guided by the results from part (a), make an educated guess of where the actual minimum $(x_1^*, x_2^*)^T$ might be (hint: If you make long enough runs, the GA will generally get very close to the actual minimum, in most runs), and then prove *analytically* (i.e. by manual calculations, without the help of a computer!) that the point $(x_1^*, x_2^*)^T$ actually is a stationary point of the function g . (You do not need to prove that it is a minimum).

What to hand in You should hand in `genetic_algorithm.py` (with all the functions added; see the list above), as well as your versions of `run_single.py` and `run_batch.py`. In your report, write a section *Problem 1.3* where, for part (a) above, you list the selected parameters (used in `run_single.py`) and also include a (neatly formatted, not a Python screen dump) table of the values of x_1 , x_2 , and (note!) $g(x_1, x_2)$ found in your 10 runs. Then, for part (b), include another neatly formatted table showing the values of p_{mut} and the median performance (of the GA) obtained over the corresponding batch runs. Also, include a plot of the median performance as a function of p_{mut} . Then, include a brief discussion of your findings related to the optimal value of p_{mut} for this problem. Finally, for part (c), write down the true minimum $(x_1^*, x_2^*)^T$, as well as the entire derivation, with all relevant intermediate steps, showing that this point is indeed a stationary point. Omitting intermediate steps may result in the solution being returned, with resulting point deductions.

Maximum number of points for this problem: 4p.

Maximum number of points if the problem must be returned for correction: 2p